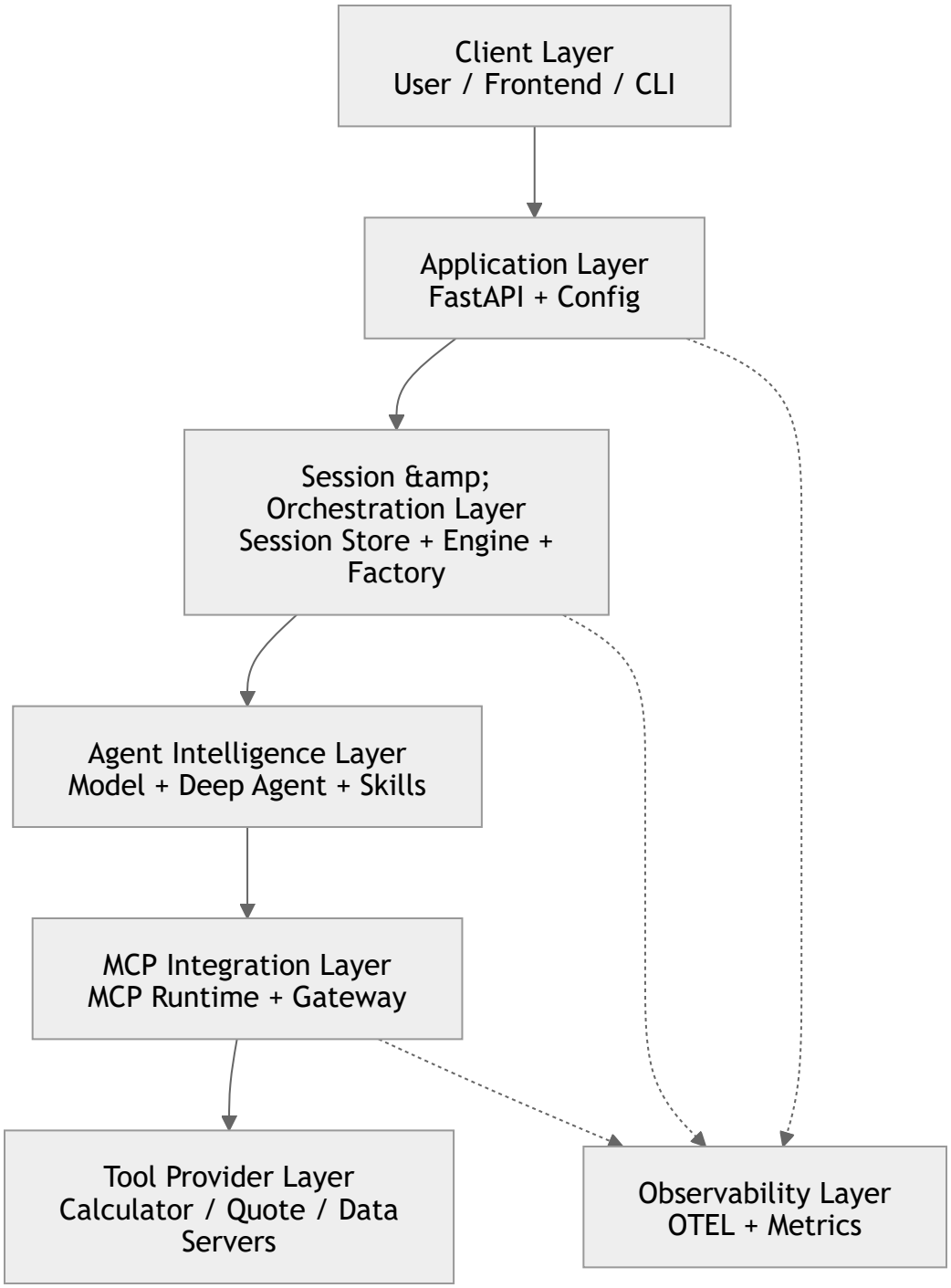


Easiest way to explain it in one sentence

You could describe the current system like this:

> **A FastAPI app receives chat requests, manages session state, invokes a single Deep Agent with skills and MCP-backed tools, routes tool access through a gateway to domain-specific MCP servers, and records traces/metrics for observability.**

If you want, I can also make a final version that is even more **presentation-friendly**, with:

- colored logical layers,
- shorter labels,
- and a cleaner “box architecture” style for slides.

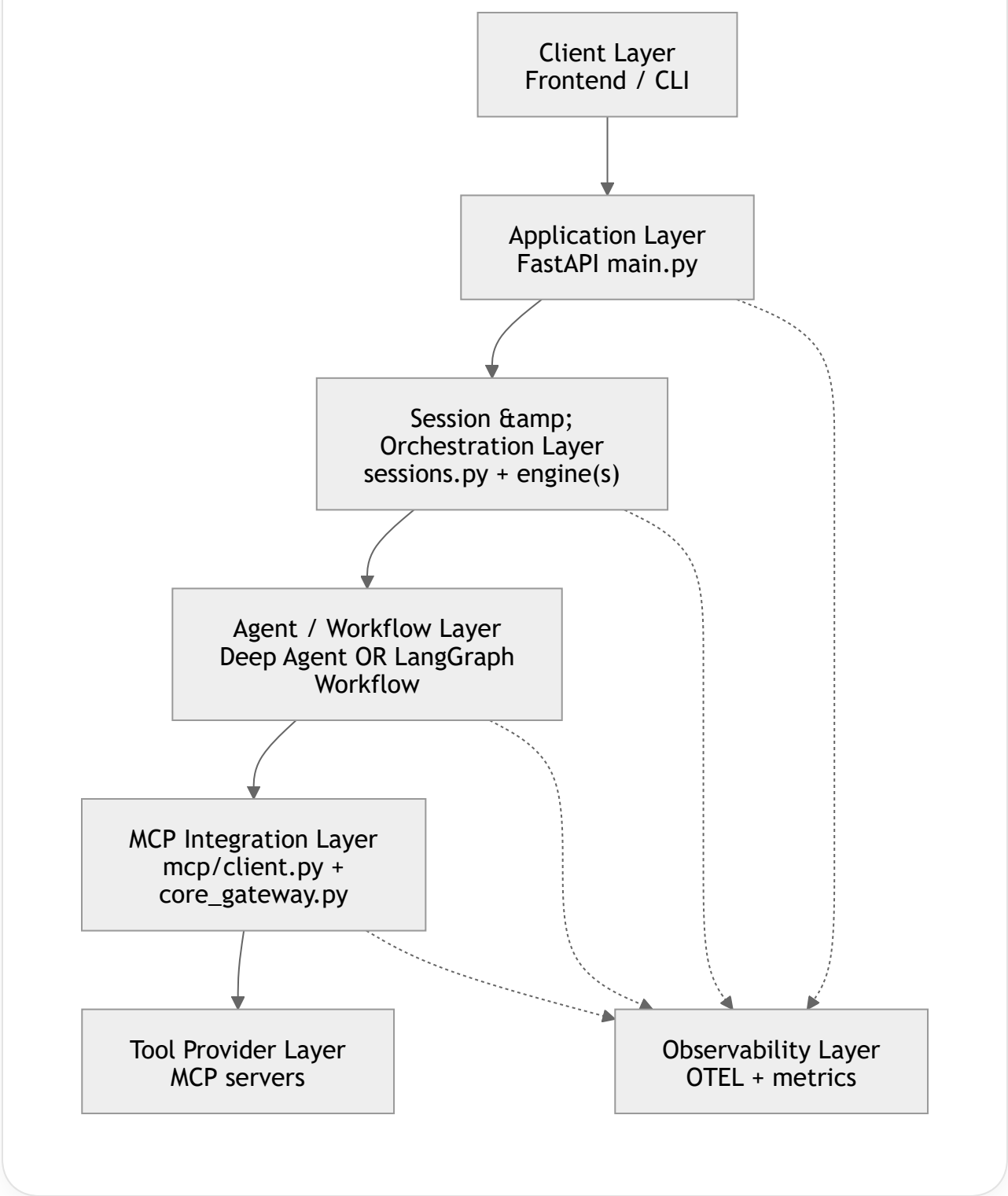
If I was going to add LangGraph workflow as another ability for my app to use, where does that plug into this architecture?

I’m checking the current LangGraph and Deep Agents docs first so I can place it in the right layer of your architecture rather than guessing from older patterns.

Thought for 33s

Yes — the cleanest place for **LangGraph workflows** to plug in is the **Session & Orchestration Layer**, not the MCP layer and not the frontend layer. LangGraph is the low-level orchestration framework for **stateful workflows and agents**, and LangChain explicitly positions it for cases where you want **fine-grained control** and to combine **deterministic and agentic steps**. Deep Agents already runs on top of LangGraph, so this is the natural layer to extend. LangChain Docs +2

So, in your architecture, I would think about it like this:



The key change is that your current `DeepAgentEngine` would stop being the **only** execution engine. Instead, the orchestration layer would choose between:

- the **Deep Agent path** for open-ended agentic tasks, and
- the **LangGraph workflow path** for fixed or semi-fixed workflows. LangGraph's own docs describe workflows as **predetermined code paths** and agents as **dynamic tool-using processes**. LangChain Docs +1

Where it fits in your code

The most practical version for your repo would be:

- keep `backend/main.py` as the API/bootstrap layer,
- keep `backend/agent/sessions.py` as the session state layer,
- add something like `backend/workflows/engine.py` or `backend/langgraph/engine.py`,
- let the session/orchestration layer choose which engine to invoke. LangGraph graphs are compiled from nodes and edges around shared state, so they fit naturally as a parallel execution engine next to your existing Deep Agent engine. LangChain Docs +1

That would look like this structurally:

```
backend/
  main.py
  agent/
    sessions.py
    engine.py              # current DeepAgentEngine
    factory.py
  workflows/
    engine.py              # new LangGraph workflow runner
    graphs/
      intake_graph.py
      data_analysis_graph.py
```

The cleanest architecture change

I would not put LangGraph "under" the MCP runtime.
I would put it **beside** `DeepAgentEngine` in the orchestration layer.

So instead of:

```
Session Store -> DeepAgentEngine -> MCP Runtime -> Gateway -> MCP Servers
```

you get:

```
Session Store
  -> DeepAgentEngine (agentic path)
  OR
  -> LangGraphWorkflowEngine (workflow path)
        -> MCP Runtime
        -> Gateway
        -> MCP Servers
```

That is because LangGraph is about **how work is orchestrated**, while the MCP runtime/gateway is still just the **tool access layer**. LangGraph nodes can call models, tools,

or other logic, but the graph itself belongs at the orchestration layer. LangChain Docs +2

Three good ways to use it

1. Parallel engine

This is the option I'd recommend first.

- `DeepAgentEngine` handles open-ended chat/tool usage.
- `LangGraphWorkflowEngine` handles fixed workflows like:
  - classify request
  - validate input
  - run data task
  - summarize result

This is the cleanest mentally because developers can trace:

- **agent path** for flexible tasks,
- **workflow path** for deterministic tasks. LangGraph is specifically recommended when you need low-level orchestration and deterministic+agentic combinations.

  LangChain Docs +1

2. LangGraph as the top-level orchestrator

You could also make LangGraph the main orchestrator and have one node call the Deep Agent.

That would look like:

```
Session Store -> LangGraph App
  -> classify node
  -> maybe Deep Agent node
  -> maybe data-processing node
  -> final response node
```

This is powerful, but it is a bigger mental leap. It makes sense if you want most of the app to become workflow-driven. LangGraph is built around named nodes connected by edges that update shared state. LangChain Docs +1

3. Deep Agent only, with more custom logic

This is the least "LangGraph-y" option. You keep Deep Agents as the center and only add LangGraph later for one special workflow. It works, but it tends to feel bolted on.

My recommendation for your app

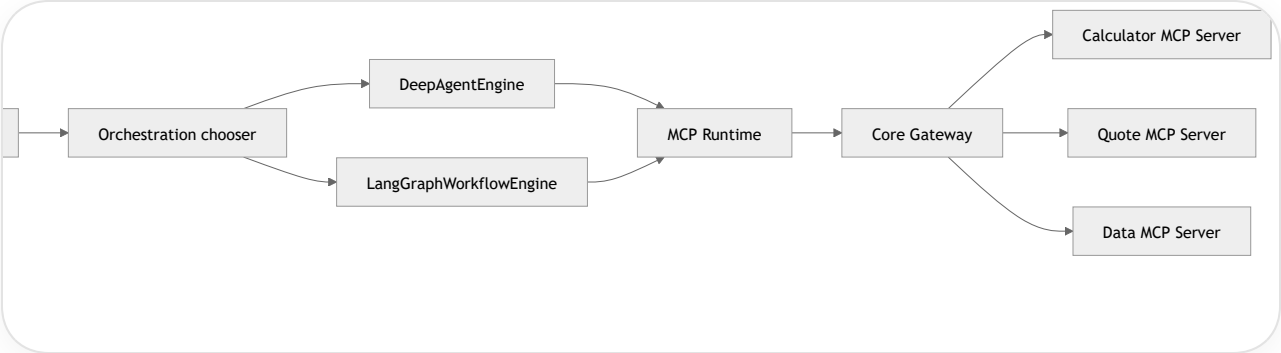
Given your current architecture, the cleanest next step is:

Add LangGraph as a second execution engine in the Session & Orchestration Layer.

That keeps the mental model simple:

- `sessions.py` still manages state and locking,
- `main.py` still wires the app,
- `mcp/client.py` and `core_gateway.py` still expose tools,
- but the app can now choose between:
  - **Deep Agent** for flexible chat/tool use,
  - **LangGraph workflow** for structured flows. This matches the official distinction between workflows and agents. LangChain Docs +1

What that would look like in a diagram



The simplest rule for choosing each path

Use **Deep Agent** when:

- the task is open-ended,
- tool choice should stay flexible,
- the user is just chatting or exploring. Deep Agents is a batteries-included harness for complex multi-step agent behavior. LangChain Docs +1

Use **LangGraph workflow** when:

- the process has known stages,
- you want explicit routing between steps,
- you need stronger control over state transitions,
- or you want deterministic and agentic steps mixed together. LangChain Docs +2

So the short answer is:

LangGraph plugs into your Session & Orchestration Layer, beside or above `DeepAgentEngine`, not below it.

The MCP runtime and gateway stay where they are; they remain shared infrastructure used by either engine. LangChain Docs +1

If you want, I can draw a **side-by-side "current vs with LangGraph" architecture diagram** next.

Sources

so how does the orchestrator choose whether to use LangGraph or Deep Agent? Is that based on the skill?

I'm checking the current official LangGraph/Deep Agents docs first, because the answer is really about where that routing decision lives in the stack.

Thought for 25s

Not by default. That choice is usually **not based on the skill**.

The cleanest way to think about it is:

- **Skills** are something the **Deep Agent** uses *after* you have already chosen the Deep Agent path. A single agent stays in control and loads specialized prompts/knowledge on demand. LangChain Docs +1
- **Choosing LangGraph vs Deep Agent** is a higher-level **routing/orchestration decision** that *your app* has to make. LangGraph's router pattern is described as a dedicated routing step that classifies input and dispatches to specialized agents or flows. LangChain Docs +1

So the order would be more like this:

```
user request
  -> app-level router / orchestrator
      -> choose Deep Agent path
      OR choose LangGraph workflow path
  -> if Deep Agent path:
         Deep Agent may then use skills
```

Why it is not naturally "based on the skill"

Because a **skill** belongs to the **single-agent pattern**:

- one agent remains in charge,
- it loads specialized context when relevant. LangChain Docs +1

A **workflow-vs-agent** choice is a different kind of decision:

- workflows have **predetermined code paths**,
- agents are **dynamic** and decide their own process/tool usage. LangChain Docs +1

So if you want the app to decide between LangGraph and Deep Agent, that decision usually lives in one of these places:

Option 1: Simple app-level router

This is the easiest to read.

Examples:

- if request matches a known structured flow, use LangGraph
- otherwise use Deep Agent

That can be based on:

- endpoint chosen by the frontend,
- a mode flag,
- simple rules,
- or one lightweight classifier step.

Option 2: LangGraph as the top-level router

This is the more “LangGraph-native” design.

You create a small LangGraph graph whose first node is a **router**:

- classify the request,
- send it to a deterministic workflow node,
- or send it to a “Deep Agent node.”

This fits LangGraph well because custom workflows can mix deterministic logic and agentic behavior, and can embed other patterns as nodes. LangChain Docs +1

Option 3: Endpoint-based separation

This is the simplest operationally.

Examples:

- `/chat` → Deep Agent

- `/workflow/data-import` → LangGraph workflow

That avoids hidden routing and is very easy for developers to trace.

What I would recommend for your app

Given your focus on readability, I would **not** make the LangGraph-vs-Deep-Agent choice based on skill matching.

I would make it an explicit **router/orchestrator decision** above the agent layer, such as:

```
ChatSessionStore
  -> OrchestratorRouter
       -> DeepAgentEngine
       -> LangGraphWorkflowEngine
```

Then:

- **DeepAgentEngine** can still use skills internally
- **LangGraphWorkflowEngine** runs explicit graphs
- the router decides which engine gets the request

That keeps the mental model clean:

- **router chooses engine**
- **Deep Agent chooses skills**
- **LangGraph chooses graph nodes/edges**

A good rule of thumb

Use **Deep Agent** when the request is open-ended:

- chat
- exploration
- flexible tool use

Use **LangGraph** when the request has a known structure:

- intake → validate → fetch → summarize
- multi-step deterministic business flow
- branching workflow with explicit states

That matches the official distinction: workflows have predetermined paths; agents are dynamic. LangChain Docs +1

So the short answer is:

No — I would not base "LangGraph or Deep Agent?" on the skill.
I would base it on a **higher-level router/orchestrator layer**, and only after the Deep Agent path is chosen would skills come into play.

If you want, I can sketch the exact file layout for adding an `OrchestratorRouter` cleanly into your current repo.

Sources

so by "router chooses engine" do you mean the LangGraph chooses which node to go next or another implementation?

I'm pulling the current official LangGraph/LangChain routing docs so I can answer this cleanly and place the decision in the right layer.

Thought for 14s

It can be **either**, but I was referring to **another implementation above the engines**, not "Deep Agent skills decide it." LangChain Docs +2

The clean distinction is:

- A **LangGraph router node** is a node inside a graph that classifies the request and chooses the next node or branch. In LangGraph, **nodes do the work** and **edges decide what runs next**, including conditional routing. LangChain Docs +1
- A **top-level app router** is your own code, outside LangGraph, that decides "send this request to `DeepAgentEngine`" or "send this request to `LangGraphWorkflowEngine`." That is just ordinary app orchestration. LangChain Docs +1

So when I said **"router chooses engine,"** I meant one of these two designs:

Option A: app-level router

Your Python app makes the choice before either engine runs.

```
FastAPI
  -> OrchestratorRouter
```

```
        -> DeepAgentEngine
        -> LangGraphWorkflowEngine
```

This is the simplest to read because the decision is explicit in your app code, and then each engine runs independently. It also keeps the mental model clean: the router is not itself a graph node; it is just the component that dispatches to one engine or the other. LangChain Docs +1

Option B: LangGraph as the top-level router

A LangGraph graph is the main orchestrator, and its first node is a router/classifier node.

```
FastAPI
  -> LangGraph app
       -> router node
            -> Deep Agent node
            -> deterministic workflow node
```

That is also valid. It uses LangGraph's normal graph model, where a router node updates state and conditional edges choose the next branch. The LangChain router pattern is explicitly described as a routing step that classifies input and dispatches to specialized agents/flows. LangChain Docs +1

So the answer is:

No, I did not specifically mean "LangGraph chooses the next node" only.
I meant **a routing layer chooses which engine to run**, and that routing layer could be:

- plain app code, or
- a LangGraph router node in a top-level graph. LangChain Docs +1

For your project, I'd recommend **Option A first** because it is easier for developers to trace:

```
main.py
  -> session store
  -> router
      -> DeepAgentEngine
      -> LangGraphWorkflowEngine
```

Then later, if you want more advanced routing, you can move that logic into a LangGraph router graph. That fits LangGraph well, but it is a bigger conceptual step. LangChain Docs +2

The clean rule of thumb is:

- **App-level router** when you want the architecture easiest to understand.
- **LangGraph router node** when you want routing itself to be part of a workflow graph.

LangChain Docs +2

If you want, I can draw the two alternatives side-by-side on your current architecture.

Sources

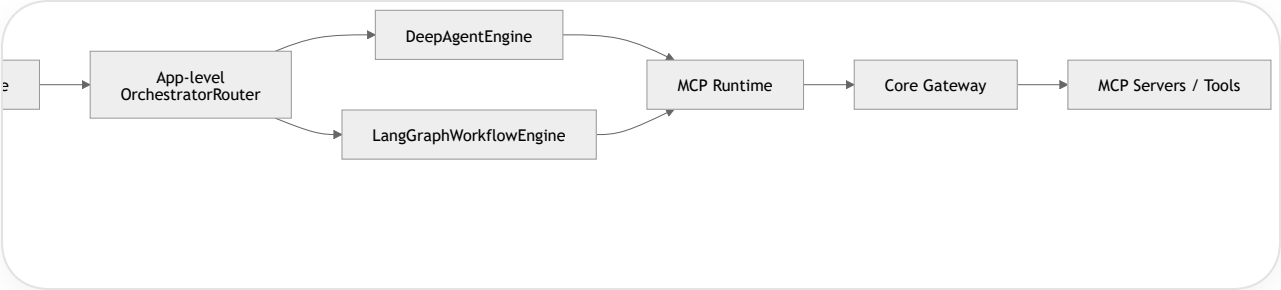
yes please

Thought for 15s

Yes — here are the two alternatives side-by-side.

Option A: app-level router outside LangGraph

This is the simpler one to read and maintain.



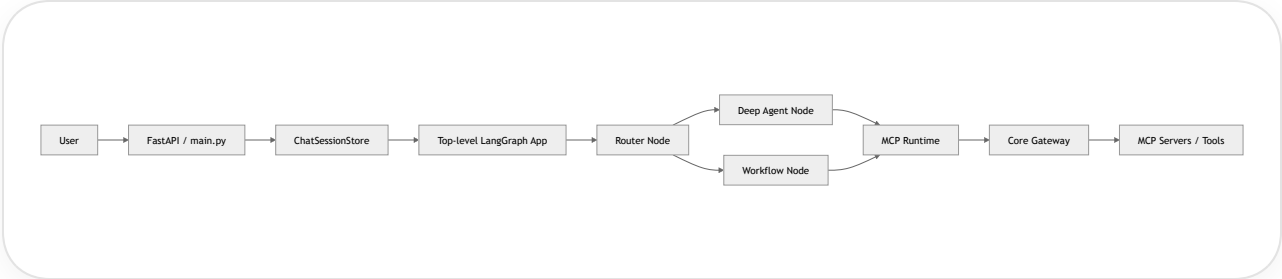
How it works:

- Your own Python router decides which engine gets the request.
- If it picks `DeepAgentEngine`, then the Deep Agent may use skills internally.
- If it picks `LangGraphWorkflowEngine`, the request enters a compiled graph/workflow. LangGraph's docs describe workflows as predetermined code paths and agents as dynamic tool-using processes. LangChain Docs

This is the easiest version for developers to trace because the dispatch is explicit in your app code.

Option B: LangGraph is the top-level router

This is the more "LangGraph-native" version.



How it works:

- The top-level engine is a LangGraph graph.
- A router node classifies the request.
- Conditional edges choose the next node or branch.
- One branch can call a Deep Agent; another branch can run a deterministic workflow. LangGraph's Graph API is built around nodes plus edges, including conditional routing, and the routing pattern is explicitly documented as classifying input and directing it to context-specific tasks. LangChain Docs +2

The key difference

Option A

The routing decision lives in **your app code**.

```
main.py / sessions.py
  -> OrchestratorRouter
      -> DeepAgentEngine
      -> LangGraphWorkflowEngine
```

Option B

The routing decision lives **inside a LangGraph graph**.

```
LangGraph
  -> router node
      -> Deep Agent node
      -> workflow node
```

Which one fits your project better

For your current project, I'd recommend **Option A first**.

Why:

- easier for less-experienced developers to follow,
- cleaner separation of responsibilities,
- simpler debugging,
- you do not need to make LangGraph the “boss” of the whole app immediately.

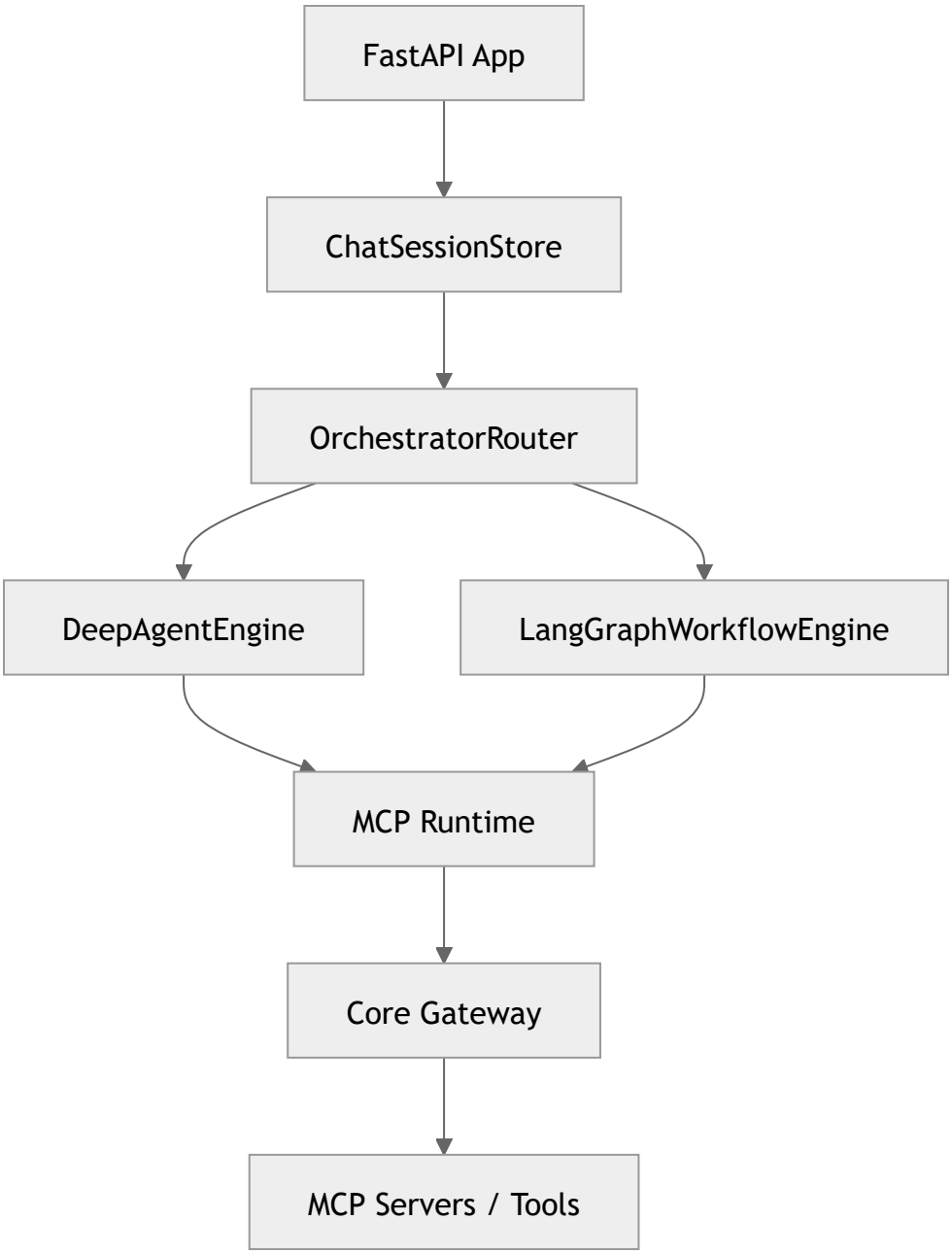
A good mental model is:

- **App router chooses engine**
- **Deep Agent chooses skills**
- **LangGraph chooses next node**

That keeps each layer’s job clear. LangGraph’s docs support using explicit routing in workflows, but that does not mean the routing has to be inside LangGraph from day one.

LangChain Docs +2

My recommendation in one diagram



This is the cleanest architecture if your priority is readability.

If you want, I can sketch what the actual file layout would look like for Option A in your current repo.

Sources

so how would an orchestratorRouter choose which engine to use? Is that the llm?

I'm checking the current LangGraph/LangChain routing docs so I can answer this in a way that matches the current patterns instead of hand-waving.

Thought for 16s ›

It can be, but it does **not have to be**.

An `OrchestratorRouter` can be implemented in three common ways:

1. **Plain code / rules**
   - Example: if the request hits a known structured flow, send it to `LangGraphWorkflowEngine`; otherwise send it to `DeepAgentEngine`.
   - LangGraph's docs explicitly frame workflows as **predetermined code paths** and agents as **dynamic** tool-using processes, so a simple rule-based split is valid. LangChain Docs +1
2. **An LLM classifier**
   - A small model call classifies the request and returns something like `"deep_agent"` or `"workflow"`.
   - LangChain's router pattern describes a routing step that **classifies input** and directs it to specialized agents or branches. LangChain Docs +1
3. **A LangGraph router node**
   - In this design, the router itself is a node inside a graph, and conditional edges decide the next node.
   - LangGraph's Graph API says **nodes do the work** and **edges determine which node executes next**. LangChain Docs

So when I said "**router chooses engine,**" I did **not** mean it must be an LLM. I meant a routing component at the orchestration layer chooses which execution path to enter.

For your app, I'd recommend this order of simplicity:

- **Start with plain code/rules**
- only add an **LLM classifier** if the split becomes fuzzy
- only make it a **LangGraph router node** if you want LangGraph to become the top-level orchestrator

That keeps it easiest for developers to read:

```
main.py
  -> ChatSessionStore
  -> OrchestratorRouter
       -> DeepAgentEngine
       -> LangGraphWorkflowEngine
```

A practical first version could be:

- requests for known structured processes → `LangGraphWorkflowEngine`
- everything else → `DeepAgentEngine`

That avoids an extra LLM call on every request while keeping the behavior easy to trace. LangChain's custom-workflow guidance also supports mixing deterministic logic with agentic behavior and embedding other architectures as nodes when needed.

LangChain Docs +1

So the direct answer is:

No, the router does not have to be the LLM.
For your codebase, I'd make it **plain application logic first**, and only later upgrade to an LLM-based classifier if you truly need smarter routing.

Sources

How would a LangGraph router node choose which path to take?

I'm pulling the current LangGraph docs for router nodes and conditional edges so I can show you the exact decision patterns, not just describe them abstractly.

Thought for 14s

A LangGraph router node chooses the path by updating or inspecting **graph state**, and then a **conditional edge function** uses that state to decide the next node. In LangGraph's Graph API, `add_conditional_edges(...)` attaches a routing function to a node; that function receives the current state and returns the next node name, a list of node names, or `END`. LangChain Docs +2

There are two common ways that happens:

1. Pure code routing

The router node or routing function uses deterministic logic. For example:

- inspect `state["intent"]`
- check whether required inputs are present
- check whether the last AI message contains tool calls
- then return `"deep_agent"`, `"workflow"`, or `END`

LangGraph's docs show exactly this pattern: a conditional edge function looks at the current state and returns the next node, and the quickstart shows a `should_continue` function that routes to a tool node if the last message has tool calls, otherwise to `END`. LangChain Docs +2

2. LLM-assisted routing

A router node can call an LLM first, store something like `state["route"] = "deep_agent"` or `state["route"] = "workflow"`, and then the conditional edge function reads that value and sends execution down the right branch. This fits LangGraph well because nodes do work and edges decide what runs next, so the node can be "classify," and the edge can branch based on the classifier result. LangChain Docs +2

So the flow usually looks like this:

```
def route_node(state):
    # deterministic or LLM-based classification
    return {"route": "workflow"}

def choose_next(state):
    return state["route"]

builder.add_node("route_node", route_node)
builder.add_conditional_edges("route_node", choose_next, {
    "workflow": "workflow_engine",
    "deep_agent": "deep_agent_engine",
})
```

The important distinction is:

- the **router node** can be simple code or an LLM call,
- the **conditional edge** is what actually selects the next path in the graph. LangGraph documents this as "nodes do the work" and "conditional edges determine which node(s) to go to next." LangChain Docs +1

For your app, the cleanest first version would usually be:

- use **plain code routing** for obvious cases,
- only use an **LLM router node** when the choice is genuinely fuzzy.

That avoids an extra model call on every request while still letting you move to smarter routing later. LangGraph explicitly separates **workflows** as predetermined paths from **agents** as dynamic processes, so this kind of top-level routing is a normal fit for the framework. LangChain Docs +1

A practical example for your architecture would be:

- if the request matches a known structured business flow, route to the LangGraph workflow branch;
- otherwise, route to the Deep Agent branch;
- if you later need more nuance, replace that deterministic router node with an LLM classifier node. LangChain Docs +1

So the direct answer is:

A LangGraph router node chooses a path by producing or exposing a routing signal in state, and a conditional edge function reads that signal to decide which branch runs next. That routing signal can come from ordinary code or from an LLM. LangChain Docs +1

Sources

can you give me how the architecture diagram would look if I used LangGraph as a top level orchestrator

I'm pulling the current LangGraph docs so I can map the top-level orchestrator pattern cleanly onto your existing layers rather than hand-waving it.

Thought for 18s

Yes — if you made **LangGraph the top-level orchestrator**, the architecture would shift from:

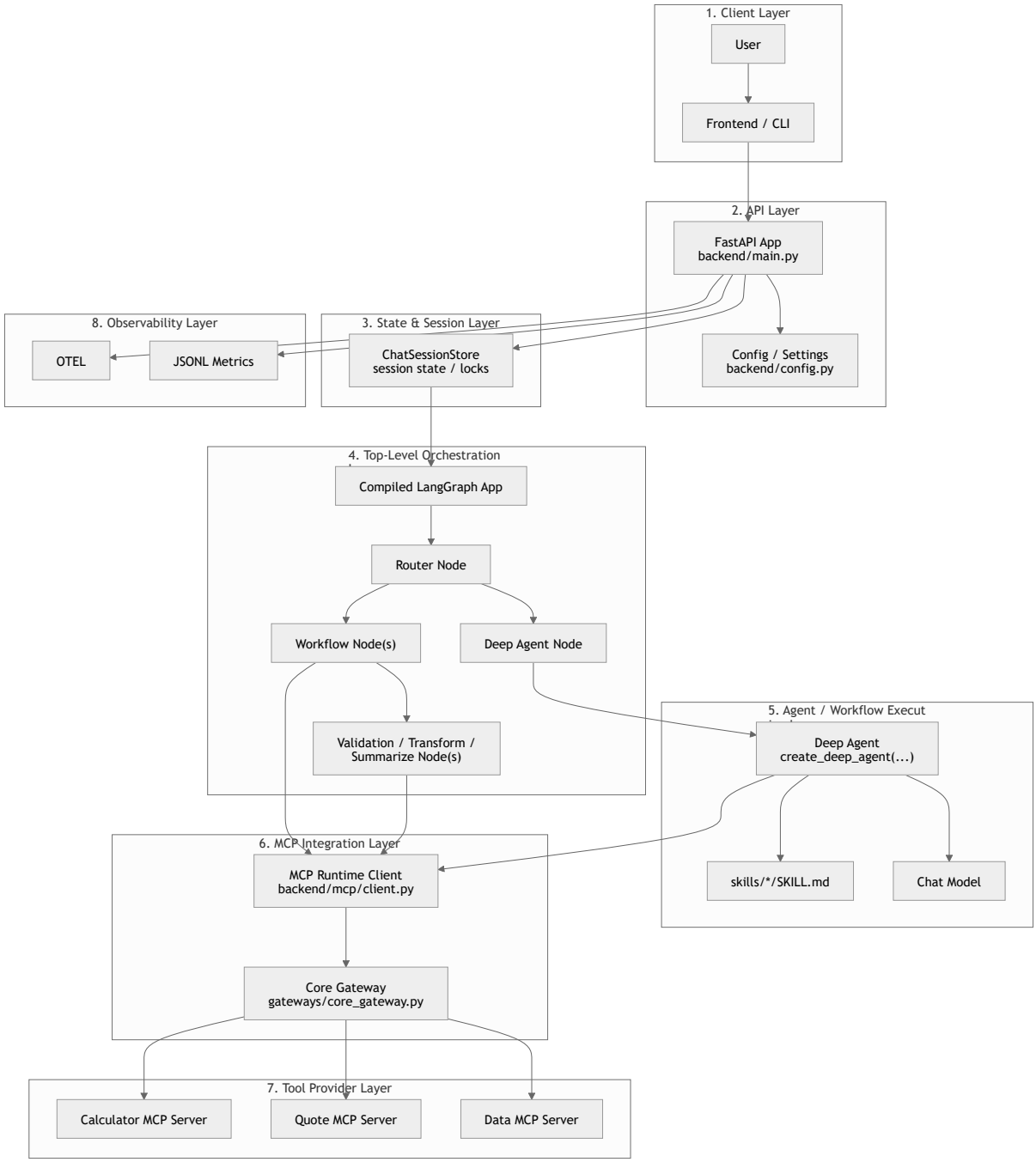
FastAPI → DeepAgentEngine → MCP runtime

to:

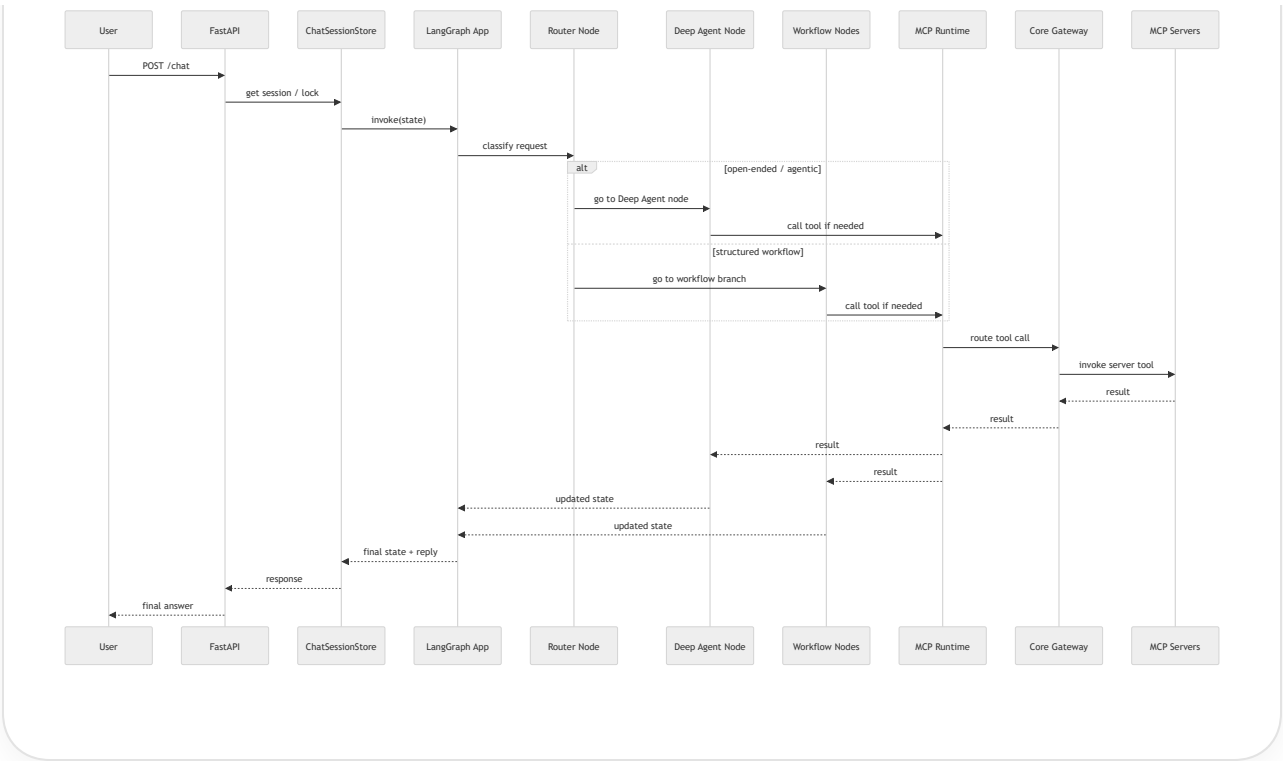
FastAPI → compiled LangGraph app → router node / workflow nodes / Deep Agent node → MCP runtime

That fits LangGraph's model directly: a graph is built from **state, nodes, and edges**, where nodes do work and edges decide what runs next. LangGraph is the low-level orchestration layer for long-running, stateful agent systems, and Deep Agents themselves run on top of LangGraph. LangChain Docs +2

Top-level LangGraph architecture



How the request would flow



What changes in your code structure

The biggest structural change is that `DeepAgentEngine` **stops being the top-level engine**. Instead, you would likely have something like:

```
backend/
  main.py
  sessions.py
  graph/
    app.py              # builds/compiles the top-level LangGraph
    state.py            # shared graph state schema
    nodes/
      router.py         # chooses branch
      deep_agent.py     # invokes create_deep_agent(...)
      workflow_data.py  # structured workflow steps
      workflow_misc.py
  mcp/
    client.py
```

That matches LangGraph's graph-first model: shared state, nodes, and edges.

LangChain Docs +1

What the router node is really doing

In this setup, the router node is just **one node in the graph**. It writes something like `state["route"] = "deep_agent"` or `state["route"] = "workflow_data"`, and then a

conditional edge sends execution to the right next node. That is the normal LangGraph routing pattern. LangChain Docs

Where Deep Agents fit now

Deep Agents would become **one branch** of the graph, not the whole app. That works cleanly because Deep Agents are built on LangGraph already, while LangGraph itself is intended for lower-level orchestration and mixing deterministic and agentic flows.

LangChain Docs +2

When this architecture makes sense

This top-level LangGraph design is strongest when you want:

- explicit routing between structured workflows and agentic behavior,
- deterministic steps mixed with tool-using steps,
- durable stateful execution,
- or human-in-the-loop pauses and resumes later. LangGraph is built specifically for durable execution, stateful orchestration, and graph-based routing. LangChain Docs +1

Tradeoff versus your current design

Your current design is easier to read:

- one app engine,
- one Deep Agent,
- one MCP runtime.

The top-level LangGraph version is more flexible, but it adds another abstraction layer because now developers have to follow:

- FastAPI,
- session store,
- LangGraph state,
- router node,
- conditional edges,
- and then either a workflow branch or a Deep Agent branch. LangGraph's docs explicitly position it as the **low-level** orchestration option, while Deep Agents are the higher-level batteries-included option. LangChain Docs +2

If you want, I can make a **side-by-side diagram of your current architecture vs. this top-level LangGraph version**.

Sources